

Targeted Scrum: Applying Mission Command to Agile Software Development

David P. Harvie, *Graduate Student Member, IEEE*, and Arvin Agah, *Senior Member, IEEE*

Abstract—Software engineering and mission command are two separate but similar fields, as both are instances of complex problem solving in environments with ever changing requirements. Our research hypothesis is that modifications to agile software development based on inspirations from mission command can improve the software engineering process in terms of planning, prioritizing, and communication of software requirements and progress, as well as improving the overall software product. Targeted Scrum is a modification of Traditional Scrum based on three inspirations from Mission Command: End State, Line of Effort, and Targeting. These inspirations have led to the introduction of the Product Design Meeting and modifications of some current Scrum meetings and artifacts. We tested our research hypothesis using a semester-long undergraduate level software engineering class. Students developed two software projects, one using Traditional Scrum and the other using Targeted Scrum. We then assessed how well both methodologies assisted the software development teams in planning and developing the software architecture, prioritizing requirements, and communicating progress. We also evaluated the software product produced by both methodologies. We found that Targeted Scrum did better in assisting the software development teams in the planning and prioritization of the requirements. However, Targeted Scrum had a negligible effect on improving the software development teams external and internal communications. Finally, Targeted Scrum did not have an impact on the product quality by the top performing and worst performing teams. Targeted Scrum did assist the product quality of the teams in the middle of the performance spectrum.

Index Terms—Scrum, Mission Command, Line of Effort, Product Design Meeting, Agile, Empirical Software Engineering

1 INTRODUCTION

WHY would military command and control (C2), and specifically mission command, be a suitable field to draw inspiration from for software engineering? Military C2 attempts to bring order and resolution to military operations which by their nature are chaotic, complex human endeavors that are constantly changing [1]. Due to their constantly changing natures, software engineering and military C2 can both be viewed as instances of complex problem solving.

Military C2 has evolved in a similar manner to software engineering. For example, military C2 and software engineering have used industrial age problem decomposition as a means to solve complex problems. The problem is analyzed, task and requirements are derived, and a scheme is developed that implements those tasks and requirements. In the Army, this is embodied by the Military Decision Making Process (MDMP) [2]. Comparing MDMP to the Waterfall Model (Table 1), it becomes apparent that there are strong similarities between traditional military problem solving and software development.

Assessment is another area of similar concern to software engineering and mission command. Software developers assess their products via validation and verification. Validation is concerned with whether the delivered software solves the right problem and meets the customer's needs. Verification is concerned with whether a specific piece of software functions according to its specified requirements [3]. Likewise, military commanders constantly assess their units' effectiveness. Commanders and their staffs determine

TABLE 1
Military Decision Making Process [2] and Waterfall Model Comparison [5].

	Military Decision Making Process	Waterfall Model
1	Receipt of Mission	System Requirements
2	Mission Analysis	Software Requirements
3	Course of Action (COA) Development	Analysis
4	COA Analysis (War Game)	Program Design
5	COA Comparison	Coding
6	COA Approval	Testing
7	Orders Production	Operations

Measures of Effectiveness (MOEs) and Measures of Performance (MOPs) in order to assess progress in a military operation. MOEs, similar to validation, ask, "Are we doing the right things?" Likewise, MOPs ask, "Are we doing things right?" [4]. These similarities of validation and verification to MOEs and MOPs suggest that software development and military communities have lessons they can learn from each other. These examples are part of the justification for looking for inspiration from mission command.

1.1 A Brief History of Mission Command

Historically, there have been two basic philosophies of command and control (C2), namely, detailed command and mission command. Detailed command is where the commander gives explicit orders to his/her subordinates on both what and how to accomplish a mission. Mission command is where the commander gives his/her intent and desired end state for a mission giving the subordinate flexibility in how to accomplish the mission [6].

Today, the United States Department of Defense (DOD) defines C2 as the commander's exercise of authority and

• D. Harvie and A. Agah are with the Department of Electrical Engineering and Computer Science, University of Kansas, Lawrence, KS, 66045.
E-mail: david.harvie@ku.edu

Manuscript received

direction of forces to accomplish a mission [4]. Mission command, which has been the U.S. Army's favored philosophy since the 1980's, has even taken on more importance in the twenty-first century [6]. General Martin Dempsey, Chairman of the Joint Chiefs of Staff, viewed the indoctrination of mission command into the United States military and he penned a white paper in 2012 to communicate the importance of mission command to future success [7]. While there may be times when detailed command is appropriate, the uncertainty and quick tempo of future military operations requires greater reliance on mission command.

1.2 Research Hypothesis

Currently, software engineering and military C2 (as expressed by mission command) are striving to be more agile and resilient to changing environments. Neither field has the panacea for complex problem solving. However, the military has developed some good techniques for responding to change due to over 11 years of continuous combat.

We have three research hypotheses regarding how our modifications to agile software development based on inspirations from mission command can improve the software engineering process. The first hypothesis is that modifications will improve the planning and prioritizing of software requirements. The second hypothesis is that the modifications will improve the communication of software requirements and progress. The third hypothesis is that the modifications will improve the overall software product.

We tested this hypothesis by developing two software projects in a classroom environment, one using a traditional agile software development methodology, Scrum, and the other using our proposed agile software development methodology, called Targeted Scrum. Our objective was to test our hypothesis for agile software development specifically to Scrum with the intent that the positive results may potentially lead to applying these inspirations to agile software development methodologies as a whole [8].

However, modifying an agile software development process is not without its tradeoffs. With a military inspiration, there tend to be a little more formalism and more meetings which may make the process feel a little less agile. However, we hypothesize that the benefits of adding those techniques will offset any perceived loss of agility.

2 BACKGROUND AND RELATED WORK

2.1 Rise of Mission Command

The first impetus for adopting Mission Command (called *Auftragstaktik* by the German Army) was Napoleon destroying the Prussian Army at the battles of Jena and Auerstedt [9]. The Prussian Army with its lock-step, rigid command structure proved extremely inadequate to face the rapidly advancing French Army. The Prussians recognized that they needed leaders who could exercise initiative and independence in order to deal with ever changing battlefield. Similarly, the United States Army experimented with Mission Command during the American Civil War. For example, General Ulysses S. Grant tasked Lieutenant General William T. Sherman with his intent of destroying one of the Confederate Armies and damaging as much Confederate war

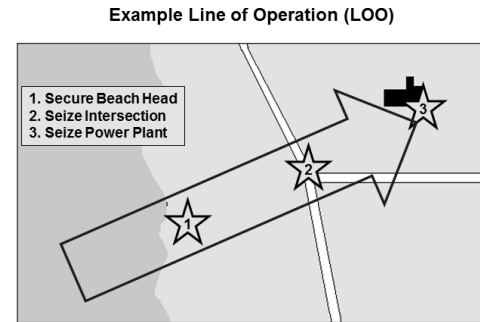


Fig. 1. Example of a Line of Operation (LOO).

resources as possible [6]. Given General Grant's intent and desired end state, Lieutenant General Sherman then had the flexibility to craft and refine his military campaign to achieve the desired end state.

2.2 Lines of Effort (LOEs)

In addition to moving towards Mission Command and away from Detailed Command, military theorists and planners sought to describe how an army can successfully conduct a military campaign. Antoine-Henri Jomini, an influential military theorist who served in Napoleon's army, emphasized the importance of lines of operations. The best chance to succeed in war was to have a unified army march along a single, safe line of operation [10]. This would allow a commander to concentrate his/her forces against the enemy before the enemy commander could do the same. The United States Army still uses the concept of line of operations (LOO) to describe how a military force moves from objective to objective. The LOO also links these objectives to the ultimate end state of the operation, and it allows the commander to visualize progress towards that end state. A LOO is formally defined as "a line that defines the directional orientation of a force in time and space in relation to the enemy and that links the force with its base of operations and objectives" [11]. Figure 1 shows a generic example of a LOO where a unit conducts an amphibious assault to secure a beach head, moves inland to seize a key road intersection, and finally moves to seize a critical power plant.

While the LOO is a great tool to describe the movement of a military force from one geographic objective to another, it is insufficient to describe how to link objectives that are not geographic in nature. The United States Army attempted to rectify this deficiency by defining logical lines of operation (LLO) in the 2001 version of Field Manual 3-0 Operations. The LLO enabled the commander to visualize objectives that are linked together by cause and effect. Even though they were established in one of the Army's capstone manuals, LLOs did not gain wide appeal until the 2006 release of Field Manual 3-24 Counterinsurgency [12].

The United States Army renamed logical lines of operations (LLOs) to lines of effort (LOEs) in the 2008 version of Field Manual 3-0 Operations [12]. As of 2011, both the U.S. Army and Department of Defense had adopted LOEs as a critical part of operational planning [11] [13]. LOEs are recognized as not only a visualization tool for a commander

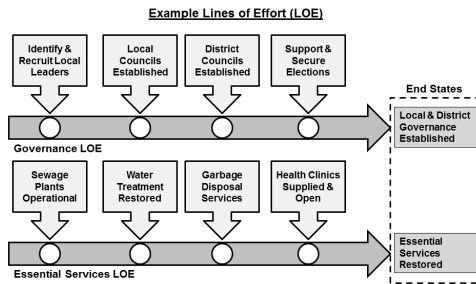


Fig. 2. Example of Lines of Effort (LOEs).

and staff to focus planning and assessments, but also LOEs are a good communication tool to other governmental and non-governmental agencies involved in the same area as the military. The LOEs allowed for unity of efforts among various agencies that may or may not have any command relationships established between them. Figure 2 shows two examples of LOEs for Governance and Essential Services. Each LOE has critical objectives that can measure progress along the specific LOE towards the LOE's end state.

2.3 Targeting and Mission Command

The heart of Mission Command is the Operations Process which is an iterative cycle of planning, preparing, executing, and continuously assessing throughout the entire process. The steps of planning, preparing, and executing are not discrete events, but instead they tend to overlap (e.g., preparing begins prior to the end of planning) [14]. A subset of this Operations Process is targeting. Targeting is formally defined as "the process of selecting and prioritizing targets and matching the appropriate response of them, considering operational requirements and capabilities" [15]. The next question then is what constitutes a target? A target is defined as "an entity (person, place, or thing) considered for possible engagement or action to alter or neutralize the function it performs for the adversary" [15]. For instance, a target could be an adversary's air defense system that must be neutralized in order to enforce a no-fly zone. Likewise, a target could be the construction of a clean water station that would give credibility to the host government. Thus, the targeting process encompasses much more than firing munitions at an adversary.

Why is there a targeting process? Targeting exists because it is not possible to detect, acquire, and service all targets at once [16]. There are real time constraints on intelligence (or knowledge), time, resource, and capabilities. Therefore, targets need to be prioritized according to how servicing that target will help the commander achieve the desired end state at a particular moment in time. Prioritization of targets naturally occurs throughout the course of an operation, therefore, an iterative targeting process is needed to properly capture those changes in priority.

The Department of Defense has formalized this process in the Joint Targeting Cycle [15] which consists of six phases:

- 1) The End State and Commander's Objectives
- 2) Target Development and Prioritization
- 3) Capabilities Analysis
- 4) Commander's Decision and Force Alignment

- 5) Mission Planning and Force Execution
- 6) Targeting Assessment

The first phase in the targeting process is to understand the ultimate desired end state for the operation and the objectives necessary in order to reach that end state. The second phase is to develop and prioritize targets that support attaining the commander's objectives. In this second phase, the prioritization is done without regard to ability to service those targets. The third phase is to determine what capabilities are available to bring to bear against the target and determine the best option, given the current state of capabilities. The fourth phase involves assigning units to service the targets based on those units' capabilities. Ultimately, the commander makes the decision to assign missions to subordinate units. The fifth phase are detailed mission planning and execution by the assigned units. The sixth phase, similar to the assessment phase in the Operations Process, is a continuous assessment of all the other phases of the Joint Targeting Cycle [15].

2.4 Overview of Scrum

Scrum is a management process, first implemented by Ken Schwaber in 1996, that can be incorporated with existing engineering processes such as software engineering. It is associated with Agile Software Development and has been used in conjunction with Extreme Programming. Scrum is based on the premise that complex activities, such as software development, are impossible to fully predict [17]. Change, therefore, is unavoidable and must be dealt with. Scrum deals with change by building software in increments versus an all-at-once approach. Scrum also makes continuous assessment throughout the building process. Finally, Scrum reviews the finished increment and makes the appropriate changes for the next increment building process.

Scrum begins with the Scrum Team, a small, cross-functional team consisting of approximately seven software engineers. The team is autonomous and self-organizing. The management representative on the team is the Scrum Master. The primary responsibilities of the Scrum Master are to remove obstacles to the team and to ensure that Scrum practices are followed [17].

Each project has a Product Backlog, which is a prioritized list of all requirements to achieve. It is an ever-changing document based on changing requirements, changing understanding of the problem, and changing environments. Only the Product Owner has the authority to prioritize the Product Backlog [17].

The management of the Product Backlog can sometimes be referred to as "grooming." Rubin [18] defines grooming as the process of managing the Product Backlog by creating and refining Product Backlog Items (PBIs), estimating the amount of work to accomplish a PBI, and prioritizing the PBIs in the Product Backlog. The grooming process keeps the Product Backlog as a relevant planning document.

Scrum breaks a project into increments called Sprints. A Sprint is a single, usually 30-day, iteration during which the Scrum Team adds new functionality to the Product. The Sprint begins with a Sprint Planning Meeting where the Product Owner, Scrum Master, and Scrum team determine

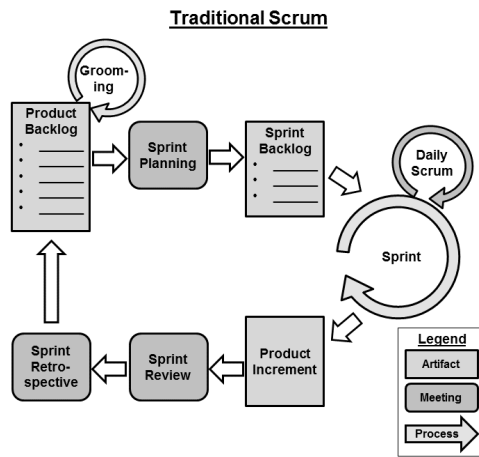


Fig. 3. Traditional Scrum Framework [18].

what portion of the Product Backlog should be developed. This portion of the Product Backlog is translated into an objective called the Sprint Goal. Then, the Scrum Team meets internally and uses the Sprint Goal to derive a list of requirements necessary to accomplish the objective. These requirements are placed into the Sprint Backlog. Success of the Sprint is dependent on whether the Sprint Goal has been achieved [17].

During the Sprint, the Scrum Master holds daily 15-minute meeting with the Scrum Team. The purpose of this meeting, called the Daily Scrum, is to review progress. Each member of the team answers three questions [19]:

- “What has been accomplished since the last meeting?”
- “What will be done before the next meeting?”
- “What obstacles are in the way?”

This entire traditional Scrum framework can be viewed in Figure 3.

The end result of the Sprint is a deliverable product increment. During a 4-hour Sprint Review, the Scrum Team delivers the product increment and presents an assessment of what they were able to accomplish during the Sprint. This assessment is compared to the Sprint Goal and Product Backlog with the purpose of updating the Product Backlog prior to the start of the next Sprint Planning meeting [17]. The Sprint Review is followed by the Sprint Retrospective which is an internal Scrum Team meeting where the team assesses its work and its processes. The purpose of the Retrospective is to improve the team’s processes, and ultimately their work, prior to the next Sprint [20].

2.5 Analysis of Scrum

The main strengths of the Scrum are its iterative process and continuous feedback. Li *et al.* [21] surveyed a company that transitioned from a plan driven software methodology to Scrum. Their survey found that the continuous daily feedback in addition to the feedback received during the sprint retrospective meetings resulted in a greater focus on software quality. Also, the daily scrum meetings fostered developer communication and interaction resulting in more team synergy. Sutherland *et al.* [22], likewise saw

similar positive aspects of Scrum from his experience with Systematic. Constant communication with the client and iterative software releases directly led to the early discovery of defects and technological issues resulting in reduced costs and scheduling issues. Customer satisfaction also increased because of the transparent process involving feedback and iterative releases.

A challenge in Scrum is the initial planning and identification of requirements. Cao & Ramesh [23] cite the neglect of nonfunctional requirements, such as scalability and maintainability, as a challenge of the iterative requirements engineering process found in agile software development. They found this challenge particularly true during the beginning stages of project development as these nonfunctional requirements were not well defined. Overhage & Schlauderer [24] cite the lack of initial planning that makes it more difficult for Scrum teams to begin a project. The team must execute a couple of sprints before the overall architecture and long-range planning begins to be established.

Another challenge in Scrum can be the Product Owner, i.e., software client representative. The client may not dedicate the most knowledgeable person to the development team because that person’s expertise is also needed by the client. This results in decreased responsiveness as the Product Owner has to reach back to find answers to the team’s questions [25] [26].

A third challenge in Scrum, closely related to the first, is the lack of focus on design. Drury *et al.* [27] report that many of the decisions made during Scrum planning were more tactical in nature (i.e., which tasks to accomplish and how to accomplish them) versus strategic (i.e., which future tasks are necessary). Very little discussion is on the design of the system as design is “emergent” based on the evolution of the system.

3 RESEARCH METHODOLOGY

3.1 Approach

Targeted Scrum attempts to provide the initial planning and overall design architecture that are missing in traditional Scrum. It is inefficient and unreasonable for the software developers to have to wait for two or three Sprint iterations before an overall architecture begins to emerge. There needs to be an initial framework of the project even if it is known that this preliminary framework can and will change. Thus, it is necessary to commit intellectual energy at the beginning of the project to establish that initial framework. The development of this framework will require the addition of an initial planning meeting prior to the creation of the Product Backlog.

Likewise, the prioritization of the Product Backlog and selection of the Product Backlog Items that are to be a part of each Sprint Backlog needs to be based on the architecture of the program. Lines of Effort (LOEs) can assist in communicating both the architecture and prioritization of the project. The Product Owner and Scrum Master can make intelligent decisions on what needs to be done next by evaluating how completing specific Product Backlog Items will advance progress along the LOEs toward the project’s ultimate end state.

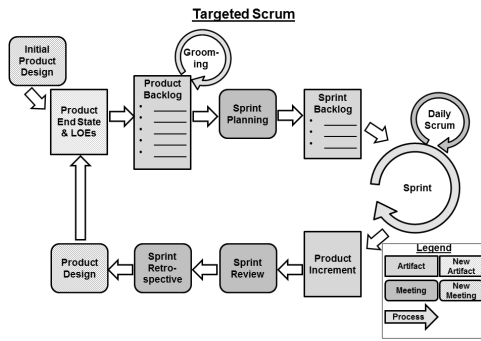


Fig. 4. Targeted Scrum Framework.

Targeted Scrum will enhance traditional Scrum in four ways:

- 1) The addition of the Product Design Meeting will establish a Product framework that is missing in traditional Scrum.
- 2) The LOEs and Product Design Meeting will establish a formal mechanism to better groom the Product Backlog.
- 3) The LOEs will improve Sprint Planning by enabling the Scrum team to select a more appropriate set of Product Backlog Items to be incorporated into the next Sprint.
- 4) The LOEs will provide a better mechanism to communicate the Product's progress to stakeholders that are both internal and external to the Scrum team.

The targeted Scrum framework is illustrated in Figure 4.

Targeted Scrum is a modification of traditional Scrum based on inspirations from Mission Command. The three inspirations of Mission Command are End State, Line of Effort, and Targeting.

3.2 Product Design Meeting

The first step in Targeted Scrum is the addition of a Product Design Meeting. The initial Product Design Meeting takes place prior to the creation of the Product Backlog. The first task of this Product Design Meeting is for the Scrum Master and the Product Owner (also referred to as the software client) to determine the end state for the project. What does the client's environment look like with the fully developed software? As stated in JP 5-0 Joint Planning, "effective planning cannot occur without a clear understanding of the end state" [13]. This end state is further detailed through the use of supporting objectives that the Product will accomplish.

The second task of the Product Design Meeting is to determine the critical software features necessary to achieve the end state and its objectives. The Product Owner and Scrum Master turn these critical features into LOEs. Each LOE has an end state, which should nest within the end state and objectives of the project. In order to reach the end state for the specific LOE, the team develops goals and milestones that are measurable and represent significant progress towards the end state. In larger software development teams, these LOEs can be the basis around which to form Scrum teams, in case the project is large enough to require multiple Scrum teams. Likewise, the LOEs can be effective tools to

TABLE 2
Agenda for the Initial Product Design Meeting.

Step	Question and/or Action
1	What is the goal/end state of the delivered product?
2	What are the features necessary to implement in order to reach the product end state?
3	Which of the identified features are absolutely critical to the product's success?
4	For each critical feature, what is the end state for that critical feature?
5	Transform each critical feature into a line of effort (LOE) with associated end state.
6	For each LOE, identify measurable milestones that mark progress towards the end state.

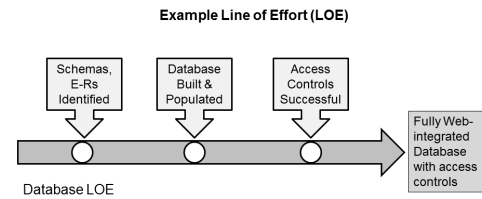


Fig. 5. Example of Database Line of Effort (LOE).

communicate progress and synchronize efforts across the Scrum teams.

The initial Product Design Meeting should last no more than two hours. The agenda of that initial meeting is outlined in Table 2.

The following is a partial example of an initial Product Design Meeting. The software client wants a Web-based personnel management system for her organization. The first step is to identify the end state for the product. In this case, the client wants a secure, efficient, and easily adaptable personnel management system for an organization of 50-200 people that can be accessed and modified via a Web browser. During the Scrum Master and Product Owner dialogue, several product features are identified, including security, Web-based interface, and personnel database. The personnel database is further identified as a critical feature. The next step is to identify the end state for the database. The end state is articulated as a fully integrated database with access controls. This critical feature with end state is then transformed into an LOE. The final step for this particular LOE is to identify measurable milestones such as identifying all schemas, entities, and relationships, populating the built database, and implementing access controls. This LOE is shown in Figure 5.

Subsequent Product Design Meetings take place after every two to three Sprints. These meetings should be limited to one hour. The purpose of these subsequent Product Design Meetings is to assess progress along the LOEs and progress towards to the overall end state. This meeting also assesses whether the product end state, objectives, and LOEs are still valid and relevant in their current forms or if they need to be modified. These assessments reflect the reality that requirements, program knowledge, and environments can and will change. The reason for holding this meeting after multiple Sprints is to allow some continuity between Sprints. The agenda for these meetings is shown in Table 3.

TABLE 3
Agenda for the Subsequent Product Design Meeting.

Step	Question and/or Action
1	Review the goal/end state of the delivered product. Is it still valid?
2	Review each LOE with its associated end state and milestones. Is each LOE still valid? Does any LOE need to be modified, added or deleted?
3	Review the progress and prioritization of the LOEs. Is the current prioritization of LOEs still valid?
4	Publish any necessary updates to the end state and/or LOEs.

3.3 Grooming the Product Backlog

In Targeted Scrum, Product Backlog grooming is an intentional conversation that is held after the Product Design Meetings. Formalizing the time and place for the Product Backlog grooming will ensure that this essential dialogue takes place on a routine basis. Also, the timing of the Product Design Meetings in relationship to subsequent Sprints will help facilitate the Scrum team to better respond to changes that result from the grooming process.

In targeting doctrine, an individual target's true importance is relative to its relationship to other targets [15]. Applying that concept to grooming, the creation and grooming of the Product Backlog can now be done against the backdrop of the Product end state, objectives, and LOEs. Product Backlog Items are identified and prioritized based on how they move the product along its LOEs towards its objectives and end state.

The LOEs provide the mechanism to formalize this dialogue as they are visual representations of the software client's priorities and desire end state. The LOEs themselves can be prioritized in some order to further communicate a shared understanding of both the desired product and the steps necessary to build that product.

3.4 Sprint Planning

In Targeted Scrum, each Sprint Planning meeting begins with a review of the Product end state, objectives, and the current progress along the LOEs. The Scrum team can then determine the most appropriate Sprint Goal to progress along the LOEs. The Scrum team can then develop the Sprint Backlog for the upcoming Sprint based on that Sprint Goal. The chosen Product Backlog Items are measured against how they will help facilitate movement along the LOEs. The program end state, objectives, and LOEs will assist the Scrum team in adhering to the overall program design when planning a Sprint. All other agenda items in the Sprint Planning meeting remain unchanged.

3.5 Sprint Review

An addition to the Sprint Review meeting in Targeted Scrum is to visually update progress along the LOEs. An example of this is shown in Figure 6. This updated progress assists the Scrum team to prepare for the next Sprint Planning meeting by communicating possible areas that they need to address next. The visualization of progress (or non-progress) makes the dialogue of the Sprint Review more

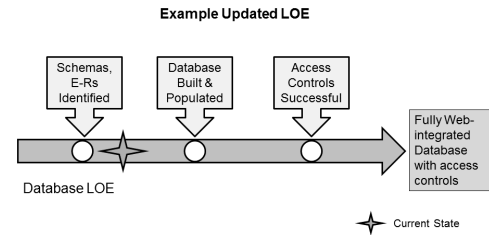


Fig. 6. Example of Updated Database LOE.

meaningful between the Product Owner and the Scrum team. Is the project progressing as anticipated? Are there concerns that need to be addressed quickly? Likewise, these updated LOEs help to succinctly communicate the product's status to the Product Owner. This is especially important if the Product Owner is a company representative who must then relay that progress to superiors who are not directly part of the Scrum process.

The LOEs and their associated milestones are considered valid at this point. The only issue to consider is the amount of progress made along the LOEs. This constrained focus keeps the Sprint Review on track. Problems uncovered with the validity of the Product end state, objectives, and LOEs should be addressed in subsequent Product Design Meetings.

4 EXPERIMENTS

4.1 Protocol

We tested the research hypothesis during a semester-long agile software engineering course designed for both graduate and upper level undergraduate students in Computer Science and Computer Engineering at the University of Kansas. We specifically designed this course to support the research. We allowed the class of 26 students to self-organize into teams of 3–4 students per team. Given the time constraints of a one-semester course, we developed and assigned two software development projects. The Sprint duration was set to two weeks, and a single project consisted of two Sprints. For the first project, half of the teams developed software using Traditional Scrum while the remaining half used Targeted Scrum. For the second project, the teams switched development methodologies. The purpose of assigning half of the teams to Traditional Scrum while the other half is assigned to Targeted Scrum is to negate the effect of students performing better on the second project from biasing the results in favor of whichever methodology is used during the second project. The instructor served as the software client for all teams in order to set and communicate a common set of objectives. We standardized the programming language for all projects to Java [28]. We encouraged the students to develop their code utilizing the Eclipse Integrated Development Environment (IDE) in order to take advantage of plugins developed for that IDE [29]. The students were informed that their code would be analyzed using the Eclipse Metrics 1.3.6 plugin [30].

For each project, there was an assigned Product Owner and a Scrum Master. At the onset of a project, the instructor gathered all the Product Owners together. Then, the instructor, acting as the client, gave the Product Owners a verbal

description of the desired software product. Some product facts, such as specific prices for goods or services were given to the Product Owners in electronic form. The Product Owners were then responsible for taking this information back to their Scrum teams. Any question that the Product Owners asked the client was answered, and the answers were communicated to the teams via electronic document on the course Web site. Upon the conclusion of the first Sprint of the project, the client would once again gather the Product Owners and insert a new requirement. This new requirement would force the Sprint teams to re-evaluate their software design and architecture.

4.2 Projects

We designed the projects so that they could be reasonably finished within a four-week time frame. We also attempted to make the difficulty level of the two projects equivalent so that we could compare the results from the two projects.

4.2.1 Project 1

Project 1 required the teams to develop a Point of Sale (POS) system for a restaurant. The POS system would keep track of employees' hours, wages, and tips. Also, the system would allow a waiter or waitress to input a customer's order. The system would also update the menu prices for items that are daily specials. Finally, the system would allow a customer to pay for the order, adding the appropriate tax and gratuity. The new requirement that we inserted at the end of Sprint 1 was the ability to reserve multiple tables for a party and allow a customer to place a deposit on that reservation.

4.2.2 Project 2

Project 2 required the teams to develop a system for a small veterinary clinic for cats and dogs. The system is required to handle veterinarian medical record information for the animal and contact/billing information for the owner. Also, the veterinary staff would use the system to schedule routine appointments and procedures. In addition, the clinic staff can execute sales transactions for both medical procedures and veterinary products. Finally, the clinic staff would use the system to schedule boarding of dogs and cats, as well as forecast weekly food requirements for their animal guests. The new requirement that we inserted at the end of Sprint 1 was the ability to send patient reminders to the animal owners regarding upcoming immunizations and appointments.

4.3 Empirical Data

In order to test whether our proposed Targeted Scrum is an improvement over Traditional Scrum, we had to gather empirical data. These data were then used to assess both the process and the delivered product. Scrum, by definition, is a process framework that is used to develop complex products such as software [19]. We had to compare the process of Targeted Scrum against the Traditional Scrum process to determine the advantages and disadvantages of using the proposed model, i.e., examine the tradeoffs.

Our empirical data consisted of four sources: (1) Software development artifacts; (2) Student surveys; (3) Instructor observations; and (4) the developed software code.

Software development artifacts such as Product Backlogs, Sprint Goals, and LOEs were necessary for both the students' academic evaluations and the understanding of how the teams determined, prioritized, and translated the project requirements into delivered code.

The second source of empirical data came from the two end of project surveys and the final comparison survey. The specific content of those surveys are outlined later. Those surveys were created using Google Forms [31].

The third source of empirical data came from instructor observations of the teams' interactions and meetings. Given that our research took place in an academic setting, the only times that students could be mandated to meet was during the class time. The teams were directed to use specific class times for the meetings directed by the Scrum methodology used, such as Sprint Planning, Daily Scrum, and Sprint Reviews. During the course of these in-class meetings, the instructor would observe and record how well the teams collaborated and communicated.

The three sources of empirical data so far described pertain solely to the process. Process, however, is not the end state for either Traditional Scrum or Targeted Scrum. Ultimately, we are developing a product — software. A “better” process that results in a “poorer” product may not actually be a better process. Thus, it was also necessary to compare the software developed by both processes. We required the teams to submit their executable code and source code as part of their academic and research evaluations.

4.4 Process Assessment

The Traditional Scrum and Targeted Scrum processes were assessed quantitatively and qualitatively using the software development artifacts, the student surveys, and the instructor observations. As part of the academic evaluation of the teams in the course, software development artifacts such as Product Backlogs, Sprint Backlogs, Product End States, and LOEs were submitted to and graded by the instructor. These artifacts were assessed on the 100-point grade scale. In order to quantitatively assess the two processes, we solely used the Product Backlogs and Sprint Backlogs since these artifacts are common to both processes.

We utilized student surveys to assess the two processes both quantitatively and qualitatively. At the conclusion of each assigned project, students were surveyed about their assessment of the software development methodology used by their team. The students' assessments were quantified using a 5-point scale with 1 representing “very poorly” or “very unlikely” to 5 representing “very well” or “very likely”. The students assessed the software development methodology that they used in four areas: (1) Project requirements; (2) Project architecture; (3) Project communication; and (4) Future use of the methodology.

Students answered multiple questions after each project to assess the methodology used regarding project requirements, project architecture, project communication, and future use of that methodology. We also asked the students to qualitatively assess the methodology by identifying two strengths and two weaknesses of that methodology.

At the end of the course, we surveyed the students a third time. The purpose of this survey was for the students

to directly compare Traditional Scrum against Targeted Scrum. The students' assessments were quantified using a 5-point scale with 1 representing "Traditional Scrum did much better" to 5 representing "Targeted Scrum did much better". The students compared the software development methodologies in four areas: project requirements, project architecture, project communication, and future use of the methodology.

Finally, we used instructor observations to qualitatively assess each team's performance with the process of software development. During the majority of these observations, the instructor did not interact with the teams. The instructor took notes on the interactions, dialogue, and actions of the teams as they collaborated on software development. However, the instructor did take on the role of the client and intentionally interacted with the teams during the project assignment with the teams' product owners and during the Sprint Review with all of the teams' members.

4.5 Product Assessment

We assessed the delivered software products by both instructor and automated evaluations. We required the teams to submit their software products in both standalone executable form and source code form. This allowed us to evaluate the teams' code similar to how a client would evaluate a software product — by actually running it. The source code was necessary in order to run the automated evaluations.

We conducted the instructor evaluations by assessing each product on (1) The product's functionality; (2) How well the product met its requirements; and (3) The aesthetics of the product. The evaluations of these three areas were then summed to yield an overall assessment of the delivered software product. We also conducted instructor evaluations on (1) Observations of how well the team employed its designated Scrum process; and (2) How maintainable is the delivered code partially based on the results of automated metrics discussed below.

We ran the automatic evaluations using the Metrics 1.3.6 plugin [30] for the Eclipse IDE. The following metrics, generated by the Metrics 1.3.6 plugin, are derived from "Object-Oriented Metrics: Measures of Complexity" [32]:

- Methods Lines of Code
- Number of Overridden Methods
- Nested Block Depth
- Lack of Cohesion of Methods
- McCabe Cyclomatic Complexity
- Weighted Methods per Class

The coupling metrics generated by the Metrics 1.3.6 plugin [30] are derived from "Agile Software Development: Principles, Patterns, Practices" [33].

The Metrics plugin can provide the following metrics:

- Afferent Coupling
- Efferent Coupling
- Instability
- Normalized Distance from the Main Sequence

In order to evaluate how well each team's code did when measured against these metrics, we ranked each team (from

TABLE 4
Averages of Project 1 Survey Responses

Question	Trad. Scrum	Targ. Scrum	Student T Test
How well did your team identify project requirements?	4.0	4.1	0.756
How well did your Scrum methodology help your team identify requirements?	4.2	4.0	0.415
How well did your team prioritize the project requirements?	3.6	3.9	0.427
How well did your Scrum methodology help your team prioritize requirements?	3.4	3.7	0.275
How well did your team define the overall project architecture?	3.1	3.8	0.118
How well did your Scrum methodology help your team develop the overall project architecture?	3.1	3.8	0.040
How well did your team communicate internally?	3.2	3.8	0.338
How well did your Scrum methodology help your team communicate internally?	3.2	3.6	0.350
How well did your team communicate externally?	3.3	2.9	0.462
How well did your Scrum methodology help your team communicate externally?	3.1	3.3	0.597
How likely are you to use this Scrum methodology in the future?	4.1	3.7	0.179
In your opinion, how well does this Scrum methodology compare to Traditional Software Engineering?	4.1	4.1	0.822
Scale: 1 = Very Poorly or Very Unlikely; 5 = Very Well or Very Likely Highlight: Targeted Scrum Performed Better Bold: 75% or higher confidence			

1 to 7) according to the specific metric. In the event of tie scores, both teams were awarded the same highest rank. We then summed the metrics rankings by team in order to generate an overall ranking of which team did the best according to metrics only.

5 EVALUATIONS AND EXPERIMENTAL RESULTS

5.1 Project Surveys

5.1.1 Project 1

For Project 1, four teams consisting of 14 students utilized Traditional Scrum to develop the software product. The remaining three teams consisting of 12 students utilized Targeted Scrum to develop the software product. All 26 students answered the end of project survey regarding how well their team identified and prioritized requirements, developed the project architecture, and communicated both internally and externally. The averages of their responses for the 12 questions, on a scale from 1 to 5, separated by Traditional Scrum and Targeted Scrum are shown in Table 4. Questions that Targeted Scrum performed better in are highlighted in light gray.

Targeted Scrum performed better in eight of the 12 questions, Traditional Scrum performed better in three of the questions, and both methodologies performed equally in one of the questions. Targeted Scrum performed better in all four questions related to how well the Scrum methodology helped the team. It is interesting that Traditional Scrum performed better in the question regarding how likely was the student to use this methodology in the future.

Students were asked to identify two strengths and two weaknesses of the Scrum methodology they used in Project 1. Students using Targeted Scrum stated that this methodology helped their teams better visualize progress and the ordering of requirements. LOEs also helped the teams communicate internally. A criticism of Targeted Scrum is the additional work to maintain the Product End State and LOEs felt like a duplication of efforts. Also, students using Targeted Scrum found difficulty in determining the correct scope for the LOE milestones. They felt that the milestones were either too broad or too narrowly focused.

5.1.2 Project 2

For Project 2, we asked the teams to switch Scrum methodologies. This meant that three teams consisting of 12 students utilized Traditional Scrum to develop the software product and four teams consisting of 14 students utilized Targeted Scrum. We repeated the same end of project survey. Once again, all 26 students answered the end of project survey. The averages of their responses are shown in Table 5. Questions that Targeted Scrum performed better in are highlighted in light gray

TABLE 5
Averages of Project 2 Survey Responses

Question	Trad. Scrum	Targ. Scrum	Student T Test
How well did your team identify project requirements?	4.4	4.4	0.797
How well did your Scrum methodology help your team identify requirements?	4.2	4.3	0.679
How well did your team prioritize the project requirements?	4.0	4.1	0.706
How well did your Scrum methodology help your team prioritize requirements?	3.7	4.1	0.201
How well did your team define the overall project architecture?	4.3	3.9	0.179
How well did your Scrum methodology help your team develop the overall project architecture?	3.8	3.9	0.546
How well did your team communicate internally?	4.3	3.6	0.086
How well did your Scrum methodology help your team communicate internally?	3.4	3.7	0.455
How well did your team communicate externally?	4.2	3.8	0.275
How well did your Scrum methodology help your team communicate externally?	3.5	3.6	0.853
How likely are you to use this Scrum methodology in the future?	4.5	4.1	0.135
In your opinion, how well does this Scrum methodology compare to Traditional Software Engineering?	4.3	4.1	0.616
Scale: 1 = Very Poorly or Very Unlikely; 5 = Very Well or Very Likely Highlight: Targeted Scrum Performed Better Bold: 75% or higher confidence			

Targeted Scrum performed better on six of the 12 questions, Traditional Scrum performed better on five of the questions, and both methodologies performed equally on one of the questions. Once again, Targeted Scrum performed better on all four questions related to how well the Scrum methodology helped the team. Likewise, Traditional Scrum

performed better on the question regarding how likely was the student to use this methodology in the future.

Again, students were asked to identify two strengths and two weaknesses of the Scrum methodology they used in Project 2. Students using Targeted Scrum stated that this methodology helped their teams better visualize progress similar to a road map. The development of end states also helped to better define what it means to be completed. A criticism of Targeted Scrum is the additional work to maintain the Product End State and LOEs was not worth the effort. Also, students using Targeted Scrum found it difficult to maintain coherency between the LOEs, Product Backlog, and Sprint Backlog.

5.1.3 Statistical Analysis

We conducted a statistical analysis of the results of these two surveys using the Student T Test function provided by Microsoft Excel 2013 [34]. We conducted a two-tailed distribution test with the null hypothesis that the two samples have equal means and equal variance. We looked for a Student T Test confidence result of ≤ 0.250 which would indicate a 75% or higher confidence that we can reject the null hypothesis that the means of the Traditional Scrum and Targeted Scrum answers are the same. We chose this confidence interval due to the limited data set that we could collect in one semester.

There were seven questions that generated results with a 75% or higher confidence that one Scrum methodology was favored over the other as shown in bold in Tables 4 and 5. The question regarding future use generated results that favored Traditional Scrum in both projects. This is an expected result as Traditional Scrum is an established methodology that many of the students have already seen implemented. The two questions regarding the prioritization of requirements and development of the overall project architecture both favored Targeted Scrum. The question, "How well did your team define the overall project architecture?", generated results that favored Targeted Scrum for Project 1 and Traditional Scrum for Project 2. The teams that used Targeted Scrum for Project 1 then used Traditional Scrum for Project 2 and vice versa. Thus, it makes sense that the teams who self-assessed themselves doing a better job in defining an overall project architecture for one project would do the same for the second project.

5.2 End of Course Survey

At the conclusion of both projects, we surveyed the students a third time with the purpose of directly comparing Traditional Scrum and Targeted Scrum. Twenty-four students (92%) responded to the survey. The first part of the survey was a direct comparison of Traditional Scrum against Targeted Scrum in terms of project requirements, project architecture, communication, and future use. The first question was which Scrum methodology did a better job in helping the team identify requirements. Three students (13%) favored Traditional Scrum, 15 students (62%) favored Targeted Scrum, and six students (25%) evaluated both methodologies as the same. The second question was which methodology performed better in facilitating the prioritization of requirements. Two students (8%) favored Traditional Scrum, 16 students (67%) favored Targeted Scrum,

and six students (25%) evaluated both methodologies as the same. In regards to both identifying and prioritizing requirements, the majority of the students assessed that Targeted Scrum did a better job in both aspects.

The second area of comparison was the development of the overall project architecture. This question asked which methodology did a better job in assisting the team to develop the project architecture. Four students (17%) assessed Traditional Scrum as better, 15 students (62%) assessed Targeted Scrum as better, and five students (21%) assessed both methodologies as the same. Similar to the results regarding requirements, a majority assessed Targeted Scrum as doing a better job in helping the teams to develop the project architecture.

The third area of comparison was in regards which methodology did a better job facilitating communication. The first question was which Scrum methodology did a better job facilitating internal communication within the team. One student (4%) assessed Traditional Scrum as better, six students (25%) assessed Targeted Scrum as better, and 17 students assessed both methodologies as the same. The second question was which Scrum methodology did a better job in facilitating external communication outside of the team. One student (4%) assessed Traditional Scrum as better in regards to external communication, eight students (33%) assessed Targeted Scrum as better, and 15 students (63%) assessed both methodologies as the same. A majority of students assessed both methodologies as being equal in facilitating both internal and external communication.

The fourth area of comparison was in regards to which Scrum methodology would students use in the future if given a choice. Seven students (29%) would choose Traditional Scrum, 15 students (63%) would choose Targeted Scrum, and two students (8%) were split on which methodology to choose. The vast majority of the students clearly favored one methodology or the other. Targeted Scrum was favored by a majority, but Traditional Scrum was favored by a very significant minority.

The end of course survey also gave students the option of providing additional comments regarding Traditional Scrum and Targeted Scrum. One-half of the students provided additional feedback, and most of that feedback were either favorable or somewhat favorable regarding Targeted Scrum. Students saw Targeted Scrum's visualization to be most beneficial in communicating progress to a management or non-technical audience. Also, they commented that if the LOEs can be printed, posted, and constantly referred to and updated then they would be a tremendous asset in assessing progress. One criticism of Targeted Scrum is that while it felt like a better method, Traditional Scrum is a great method too. It was difficult to distinguish the two. Also, the additional overhead associated with Targeted Scrum could either be minimal or very burdensome depending on the team members.

5.3 Project Grades

Up to this point, the evaluations have focused solely on the processes of Traditional Scrum and Targeted Scrum. We also evaluated the software produced by both methodologies to test our hypothesis that Targeted Scrum will improve the

TABLE 6
Project 1 Grade Rankings by Methodology and Team

Methodology	Team	Fun.	Req.	Aes.	Overall
Traditional	A	6	2	3	3
	B	4	4	6	5
	C	2	7	7	7
	D	2	4	3	4
Targeted	E	1	1	1	1
	F	6	2	1	2
	G	4	6	5	5

Fun. – Functionality; Req. – Requirements; Aes. – Aesthetics

quality of the delivered product. A team's project grade for the class consisted of:

- The instructor's assessment of the team use of the Scrum methodology
- An assessment of the delivered software's basic functionality
- An assessment of how well the delivered software met customer requirements
- An assessment of the software's aesthetics
- An automated assessment of the code quality using the Metrics plugin
- An assessment of the team's presentation of their delivered software

To test our research hypothesis, we used the assessments of the programs' basic functionality, meeting of requirements, and overall aesthetics to determine the quality of the program. We then summed those assessments to give an overall assessment. We then translated each of these four scores into rankings among the teams from one to seven, with one representing the highest evaluation in a category. The automated metrics will be discussed later in §5.4.

5.3.1 Project 1

Project 1 grade rankings are shown in Table 6. The top two overall teams, E and F, both used Targeted Scrum, and the third and fourth place overall teams, A and D, used Traditional Scrum. Since our research hypothesis focused on the identification and prioritization of requirements, we also wanted to pay attention to how well each team's software product met the customer's requirements. The overall best team in regards to requirements, E, used Targeted Scrum, and two teams, A and F, tied for second place with team A using Traditional Scrum and F using Targeted Scrum.

5.3.2 Project 2

Project 2 grade rankings are shown in Table 7. One Targeted Scrum team, A, and one Traditional Scrum team, E, both claimed the top overall team ranking and top ranking in regards to meeting requirements. Teams B and D, which both used Targeted Scrum, claimed the third and fourth positions in relation to both overall score and meeting requirements, respectively.

5.3.3 Trends

We looked at both project grade rankings in order to identify any similarities. Team E produced the best software for both projects, and team C produced the worst software for both projects. This suggests that the best and worst

TABLE 7
Project 2 Grade Rankings by Methodology and Team

Methodology	Team	Fun.	Req.	Aes.	Overall
Targeted	A	4	1	1	1
	B	4	3	4	3
	C	3	7	6	7
	D	1	4	6	4
Traditional	E	4	1	1	1
	F	7	4	1	5
	G	1	6	4	6

Fun. – Functionality; Req. – Requirements; Aes. – Aesthetics

TABLE 8
Changes in Team Grade Rankings from Project 1 to Project 2

Traditional Scrum to Targeted Scrum				
Team	Fun.	Req.	Aes.	Overall
A	+2	+1	+2	+2
B	0	+1	+2	+2
C	-1	0	-1	0
D	+1	0	-3	0
Sum	+2	+2	0	+4
Targeted Scrum to Traditional Scrum				
Team	Fun.	Req.	Aes.	Overall
E	-3	0	0	0
F	-1	-2	0	-3
G	+3	0	-1	-1
Sum	-1	-2	-1	-4

Fun. – Functionality; Req. – Requirements;
Aes. – Aesthetics

performing teams will continue to do the best or worst no matter which Scrum methodology is used. We then compared each team's rankings going from Project 1 to Project 2 in order to identify any changes going from one Scrum methodology to the other. Those changes in grade rankings are displayed in Table 8. Two teams, A and B, saw improvement in overall score and meeting requirements when going from Traditional Scrum to Targeted Scrum. One team, F, saw decreased performance in overall score and meeting requirements when going from Targeted Scrum to Traditional Scrum. The remaining four teams' ability to meet requirements and overall performances practically remained the same under both Scrum methodologies. These comparisons suggest that Targeted Scrum can assist teams in meeting requirements and producing better software, but it is by no means a guarantee.

5.4 Product Metrics

We used the Metrics 1.3.6 plugin [30] to generate automated metrics for the source code developed by the teams.

5.4.1 Project 1

The automated metrics for Project 1 are displayed in Table 9. The values in this table are averages calculated by the Metric plugin when analyzing every class. Lower metric numbers generally represent better written code (e.g., lower complexity and lower instability) with the exceptions of two categories, Number of Overridden Methods and Afferent Coupling.

In order to get a more holistic assessment of the code quality, we translated the raw metrics into rankings per metric from one (the team performing the best according to that

TABLE 9
Automated Metrics Generated for Project 1

Metric	Traditional Scrum				Targeted Scrum		
	A	B	C	D	E	F	G
MLOC	34.390	33.140	2.701	23.690	15.936	28.204	11.771
NBD	2.259	2.140	2.259	2.379	1.521	1.861	1.403
NORM	0.000	0.000	0.250	0.000	0.000	0.034	0.000
MCC	2.789	2.047	1.102	2.879	1.457	1.964	1.604
WMC	17.083	6.286	9.750	13.917	11.393	9.276	16.500
AC	1.000	0.000	3.571	0.000	5.429	0.000	1.500
EC	2.000	4.000	1.571	0.000	2.714	2.000	1.500
Inst	0.500	1.000	0.288	1.000	0.468	1.000	0.500
NDM	0.500	0.000	0.640	0.000	0.532	0.000	0.500
LCOM	0.448	0.352	0.608	0.196	0.172	0.404	0.463

MLOC – Method Lines of Code; NBD – Nested Block Depth;
NORM – Number of Overridden Methods;
MCC – McCabe Cyclomatic Complexity
WMC – Weighted Methods per Class; AC – Afferent Coupling;
EC – Efferent Coupling; Inst – Instability
NDM – Normalized Distance from Main;
LCOM – Lack of Cohesion of Methods;

TABLE 10
Automated Metrics Rankings for Project 1

Metric	Traditional Scrum				Targeted Scrum		
	A	B	C	D	E	F	G
MLOC	7	6	1	4	3	5	2
NBD	5	4	5	7	2	3	1
NORM	3	3	1	3	3	2	3
MCC	6	5	1	7	2	4	3
WMC	7	1	3	5	4	2	6
AC	4	5	2	5	1	5	3
EC	4	7	3	1	6	4	2
Inst	3	5	1	5	2	5	3
NDM	4	1	7	1	6	1	4
LCOM	5	3	7	2	1	4	6
Sum of Ranks	48	40	31	40	30	35	33
Overall Rank	7	5	2	5	1	4	3

metric) to seven (the team performing the worst according to that metric). We then summed the metric rankings per team to get an overall score. The lowest score represented the team with the best code quality according to the Metrics plugin. We then ranked the teams according to this overall score from one (the best) to seven (the worst). The results of these metrics rankings for Project 1 are shown in Table 10. The Targeted Scrum teams (E, G, F) did very well by placing first, third, and fourth places, respectively.

5.4.2 Project 2

We then repeated this process of generating automated metrics and subsequent rankings for Project 2. The raw metrics for Project 2 are shown in Table 11, and the rankings are displayed in Table 12. The performance of the Targeted Scrum teams (C, A, D, and B) was mixed as they placed first, second, fifth, and seventh, respectively.

5.4.3 Trends

We then looked at how the teams' overall rankings did or did not change as they transitioned from one Scrum Methodology to the other. These changes are shown in Table 13. For the four teams that used Traditional Scrum for Project 1 and Targeted Scrum for Project 2, the results were mixed. Team A saw dramatic code improvement. Team B

TABLE 11
Automated Metrics Generated for Project 2

Metric	Targeted Scrum				Traditional Scrum		
	A	B	C	D	E	F	G
MLOC	13.336	34.050	3.246	29.467	15.937	26.890	18.031
NBD	1.576	1.934	1.101	2.533	1.611	1.911	1.214
NORM	0.922	0.087	0.250	0.000	0.000	0.070	0.000
MCC	2.134	2.678	1.076	3.950	1.971	2.906	1.389
WMC	28.781	14.087	9.281	13.167	16.821	12.907	16.545
AC	5.000	0.000	5.000	0.000	4.875	0.000	1.000
EC	5.600	3.000	2.571	0.000	2.625	7.000	1.000
Inst	0.644	1.000	0.470	1.000	0.435	1.000	0.667
NDM	0.322	0.000	0.473	0.000	0.524	0.000	0.333
LCOM	0.475	0.528	0.592	0.165	0.292	0.175	0.308

TABLE 12
Automated Metrics Rankings for Project 2

Metric	Targeted Scrum				Traditional Scrum		
	A	B	C	D	E	F	G
MLOC	2	7	1	6	3	5	4
NBD	3	6	1	7	4	5	2
NORM	1	3	2	5	5	4	5
MCC	4	5	1	7	3	6	2
WMC	7	4	1	3	6	2	5
AC	1	5	1	5	3	5	4
EC	6	5	3	1	4	7	2
Inst	3	5	2	5	1	5	4
NDM	4	1	6	1	7	1	5
LCOM	5	6	7	1	3	2	4
Sum of Ranks	36	47	25	41	39	42	37
Overall Rank	2	7	1	5	4	6	3

experienced a slight decline, while Team C experienced a slight improvement. Team D remained consistent. For the three teams that used Targeted Scrum for Project 1 and Traditional Scrum for Project 2, they generally saw a decline in code quality. Team E and Team F both declined. Team G remained consistent. These comparisons may suggest some slight code improvement in teams using Targeted Scrum. However, it is important to note that automated metrics show only a partial view of the code quality as these metrics do not measure how well the code meets a customer's requirements. A clear example of this is Team C. Team C did consistently the best (second and first places) in code according to the automated metrics. However, Team C also performed consistently the worst (see Tables 6 and 7) in developing code that met customer requirements and overall software product. Thus, it is important to treat automated metrics as simply partial, alternative views of the quality of the written code.

5.5 Sample Student Product End States and LOEs

The following are sample student Product End States and LOEs created and submitted by teams using Targeted Scrum. The samples displayed represent the best submitted Product End State and set of LOEs for each project.

5.5.1 Project 1

Project 1 required the teams to develop a Point of Sale (POS) system for a restaurant. One team developed the following as their Product End State:

An intuitive, reliable point of sale system for restaurants that integrates order tracking, payment

TABLE 13
Changes in Team Metrics Rankings from Project 1 to Project 2

Traditional to Targeted	
Team A	+5
Team B	-2
Team C	+1
Team D	0
Sum	+4
Targeted to Traditional	
Team E	-3
Team F	-2
Team G	0
Sum	-5

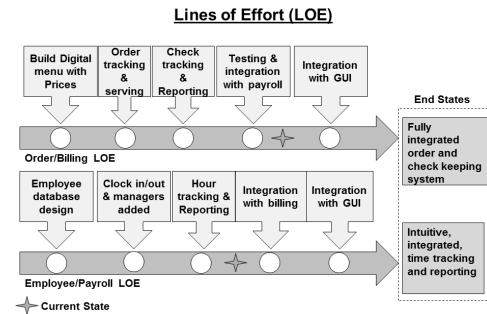


Fig. 7. Sample of Project 1 LOEs.

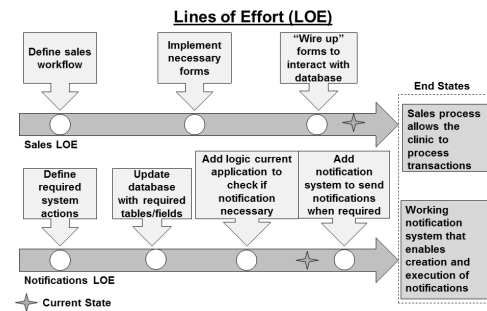


Fig. 8. Sample of Project 2 LOEs.

processing, and employee time tracking controlled with a simple interface.

They developed four LOEs to support this Product End State. Figure 7 shows two of those four LOEs.

5.5.2 Project 2

Project 2 required the teams to develop a system for a small veterinary clinic. One team developed the following as their Product End State:

A functioning application that is intuitive to use and allows the pet clinic to create new services, boarding, and sales tasks that enables the pet clinic to efficiently complete daily tasks. A customizable notification system alerts customers of upcoming appointments/needs.

They developed six LOEs to support this Product End State. Figure 8 shows two of those six LOEs.

6 CONCLUSION

6.1 Theoretical Contributions

Targeted Scrum has contributed to the enhancement of Traditional Scrum in the following ways:

- 1) Our modification of Scrum using mission command as inspiration is novel in approach and allows for future collaboration between the software development and mission command communities.
- 2) We added the Product Design meeting to initially establish the Product framework and to periodically update that framework as necessary.
- 3) We introduced end state and lines of effort (LOEs) as mechanisms to establish and maintain the Product framework.
- 4) We used LOEs to communication prioritization of the Product Backlog Items which assist in the development of the Sprint Goals and Sprint Backlogs.
- 5) We used LOEs to communicate progress in developing the Product to stakeholders both internal to and external to the Scrum team.

6.2 Findings

We had three research hypotheses regarding modifications to agile software development based on inspirations from mission command, called Targeted Scrum. The first hypothesis was that Targeted Scrum would improve the planning and prioritization of software requirements. Our finding is that Targeted Scrum did help improve the planning and prioritization of software requirements. In the two Project surveys, students consistently favored Target Scrum in the identification and prioritization of requirements, as well developing the overall project architecture. In the Comparison survey, the students clearly preferred Targeted Scrum over Traditional Scrum to identify and prioritize software requirements.

The second hypothesis was that Targeted Scrum would improve the communication of software requirements and progress. Our finding is that Targeted Scrum had a negligible improvement on communication. In the Project Surveys, the teams' self-assessments of their internal communication were split between both methodologies while the self-assessments of their external communication favored Traditional Scrum. It is interesting to note that the same students ranked Targeted Scrum of doing a better job in assisting them communicate both internally and externally. In the Comparison survey, the vast majority of students ranked Traditional Scrum and Targeted Scrum as being equal in their ability to communicate internally and externally. A change in the experiment parameters, such as larger Scrum teams or longer project times, may be needed to accentuate the impact of Targeted Scrum on communication.

The third hypothesis was that Targeted Scrum would improve the overall delivered software product. Our finding is that Targeted Scrum had some improvement on the final delivered product. In the graded assessments of meeting requirements and overall performance, all the teams that used Traditional Scrum for Project 1 and then transitioned to Targeted Scrum for Project 2 saw either improvement in performance or stayed the same. Conversely, all the teams

that used Targeted Scrum for Project 1 and then transitioned to Traditional Scrum for Project 2 either stayed the same or decreased in performance. It is important to note that the top performing and worst performing teams were consistent for both projects suggesting that choice of Scrum methodology had no effect on the ends of the spectrum. The automated metrics provided more mixed results. For the teams that used Traditional Scrum for Project 1 and then transitioned to Targeted Scrum, one team saw significant improvement while the remaining three either stayed the same or changed slightly. The teams that used Targeted Scrum for Project 1 and then transitioned to Traditional Scrum either remained the same or dropped two to three spots in the metrics rankings.

6.3 Limitations and Future Work

While we were able to successfully test our research hypothesis, it is important to note the limitations of our experiments. Further research should be based on trying to remove these limitations. The three major limitations of our research were the academic environment, the academic calendar, and the limited data set.

The first major limitation is that we conducted our experiments in an academic versus industrial setting. We could only guarantee that the Scrum teams met twice a week during class time. Teams using Scrum should be meeting and working together on a daily basis. Another difficulty with the academic setting is the shared classroom. The students had to sterilize their areas after each class. This prevented the students from posting their LOEs or notes on the white board.

The second major limitation is the academic calendar. We were limited to a one-semester course to both teach Traditional Scrum and Targeted Scrum and run two projects for our experiments. This meant that each project was limited to one month. As a result, we had to decrease the Sprint time from the typical four weeks to two. The decreased Sprint time placed additional overhead on the students for the work that they were doing. Likewise, the one-month project time limit meant that the project itself could not be too difficult.

The third major limitation is the limited data set. This research was conducted over one semester with 26 students which provided great insights into Targeted Scrum. However, repeated experiments with more software developers are needed to validate our initial results. Also, the data set derived from a survey of 12 questions is somewhat limited so a survey with more questions would provide a more robust data set to analyze.

ACKNOWLEDGMENTS

This article reflects the views of the authors and does not reflect the official policies or views of the U.S. Army or United States Military Academy.

REFERENCES

- [1] U.S. Department of the Army, *Army Doctrine Publication 6-0 Mission Command*, Washington, DC, May 2012.
- [2] —, *Army Doctrine Reference Publication 5-0 The Operations Process*, Washington, DC, May 2012.

- [3] "Systems and software engineering – vocabulary," *ISO/IEC/IEEE 24765:2010(E)*, pp. 1–418, Dec 2010.
- [4] U.S. Department of Defense, *Joint Publication 3-0 Joint Operations*, Washington, DC, Aug. 2011.
- [5] W. W. Royce, "Managing the development of large software systems," in *Proceedings of IEEE WESCON*. New York, NY: IEEE Computer Society Press, Aug. 1970, pp. 1 – 9.
- [6] U.S. Department of the Army, *Army Doctrine Reference Publication 6-0 Mission Command*, Washington, DC, May 2012.
- [7] M. E. Dempsey, "Mission command white paper," Chairman, Joint Chiefs of Staff, Tech. Rep., Apr. 2012. [Online]. Available: http://www.jcs.mil/content/files/2012-04/042312114128_CJCS_Mission_Command_White_Paper_2012_a.pdf
- [8] D. P. Harvie and A. Agah, "Conceptual design of targeted scrum: Applying mission command to agile software development," in *C2 Agility: Lessons Learned from Research and Operations*, ser. 19th International Command and Control Research and Technology Symposium. Alexandria, VA: Command and Control Research Program, US DOD, June 2014. [Online]. Available: http://www.dodccrp.org/events/19th_icrts_2014/post_conference/papers/006.pdf
- [9] M. G. W. Widder, "Auftragstaktik and innere führung: Trademarks of german leadership," *Military Review*, vol. 82, no. 5, pp. 3–9, Sep./Oct. 2002.
- [10] J. Shy, "Jomini," in *Makers of Modern Strategy from Machiavelli to the Nuclear Age*, P. Paret, G. A. Craig, and F. Gilbert, Eds. Princeton, NJ: Princeton University Press, 1986, ch. 6, pp. 143–185.
- [11] U.S. Department of the Army, *Army Doctrine Publication 3-0 Unified Land Operations*, Washington, DC, May 2012.
- [12] D. H. Edwan, "A symbolic interpretation of the lines of effort through the theory of strategic ambiguity," Master's thesis, School of Advanced Military Studies, Command and General Staff College, Fort Leavenworth, KS, 2009. [Online]. Available: <http://cgsc.contentdm.oclc.org/cdm/ref/collection/p4013coll3/id/2516>
- [13] U.S. Department of Defense, *Joint Publication 5-0 Joint Operation Planning*, Washington, DC, Aug. 2011.
- [14] U.S. Department of the Army, *Army Doctrine Publication 5-0 The Operations Process*, Washington, DC, May 2012.
- [15] U.S. Department of Defense, *Joint Publication 3-60 Joint Targeting*, Washington, DC, Jan. 2013.
- [16] U.S. Department of the Army, *Field Manual 3-60 The Targeting Process*, Washington, DC, Nov. 2010.
- [17] K. Schwaber and M. Beedle, *Agile Software Development with Scrum*. Upper Saddle River, NJ: Prentice Hall, 2002.
- [18] K. S. Rubin, *Essential Scrum: A Practical Guide to the Most Popular Agile Process*. Upper Saddle River, NJ: Addison-Wesley, 2012.
- [19] K. Schwaber and J. Sutherland, "The scrum guide, the definitive guide to scrum: The rules of the game," Oct. 2011. [Online]. Available: http://www.scrum.org/Portals/0/Documents/ScrumGuides/Scrum_Guide.pdf
- [20] —, *Software in 30 Days: How Agile Managers Beat the Odds, Delight Their Customers, and Leave Competitors in the Dust*. Hoboken, NJ: John Wiley & Sons, 2012.
- [21] J. Li, N. B. Moe, and T. Dybå, "Transition from a plan-driven process to scrum: a longitudinal case study on software quality," in *Proceedings of the 2010 ACM-IEEE International Symposium on Empirical Software Engineering and Measurement*, ser. ESEM '10. New York, NY, USA: ACM, 2010, pp. 13:1–13:10. [Online]. Available: <http://doi.acm.org/10.1145/1852786.1852804>
- [22] J. Sutherland, C. Jakobsen, and K. Johnson, "Scrum and cmmi level 5: The magic potion for code warriors," in *Agile Conference (AGILE)*, 2007, Aug. 2007, pp. 272–278.
- [23] L. Cao and B. Ramesh, "Agile requirements engineering practices: An empirical study," *Software, IEEE*, vol. 25, no. 1, pp. 60–67, Jan 2008.
- [24] S. Overhage and S. Schlauderer, "Investigating the long-term acceptance of agile methodologies: An empirical study of developer perceptions in scrum projects," in *System Science (HICSS)*, 2012 45th Hawaii International Conference on, Jan. 2012, pp. 5452–5461.
- [25] E. Hochmüller, "The requirements engineer as a liaison officer in agile software development," in *Proceedings of the 1st Workshop on Agile Requirements Engineering*, ser. AREW '11. New York, NY, USA: ACM, 2011, pp. 2:1–2:4. [Online]. Available: <http://doi.acm.org/10.1145/2068783.2068785>
- [26] R. Hoda, P. Kruchten, J. Noble, and S. Marshall, "Agility in context," in *Proceedings of the ACM international conference on Object oriented programming systems languages and applications*, ser. OOPSLA '10. New York, NY, USA: ACM, 2010, pp. 74–88. [Online]. Available: <http://doi.acm.org/10.1145/1869459.1869467>
- [27] M. Drury, K. Conboy, and K. Power, "Obstacles to decision making in agile software development teams," *Journal of Systems and Software*, vol. 85, no. 6, pp. 1239 – 1254, 2012, special Issue: Agile Development. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S0164121212000374>
- [28] Oracle, "Java," 2013. [Online]. Available: <http://www.java.com>
- [29] The Eclipse Foundation, "Eclipse," 2013. [Online]. Available: <http://www.eclipse.org>
- [30] F. Sauer, "Metrics 1.3.6," April 2013. [Online]. Available: <http://metrics.sourceforge.net>
- [31] Google, "Google docs," 2014. [Online]. Available: <http://docs.google.com>
- [32] B. Henderson-Sellers, *Object-oriented Metrics: Measures of Complexity*, ser. Prentice Hall Object-Oriented Series. Upper Saddle River, NJ: Prentice Hall PTR, 1996.
- [33] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*. Upper Saddle River, NJ: Pearson Education, Inc., 2003.
- [34] Microsoft, Inc., "Excel 2013," 2103.



David P. Harvie earned a B.S. in computer science from the United States Military Academy and a M.S. in computer science from North Carolina State University. He is a Ph.D. candidate at the University of Kansas and a lieutenant colonel in the U.S. Army. His research interests include software engineering, network security, and data mining. He is a graduate student member of IEEE and ACM.



Arvin Agah Dr. Arvin Agah is Associate Dean for Research and Graduate Programs in the School of Engineering and Professor of Electrical Engineering and Computer Science at the University of Kansas. His research interests include applied artificial intelligence and autonomous robots. He has edited two books and published more than 160 refereed articles in these areas. Dr. Agah has been a co-investigator on projects that represent more than \$33 million in research funding. He has supervised 15 Ph.D.

students and 39 M.S. students and has received multiple honors for his teaching excellence. Prior to joining the University of Kansas, he spent two years at the Bio-Robotics Division of Mechanical Engineering Laboratory in Tsukuba, Japan. He has been a researcher at the IBM Los Angeles Scientific Center, and Xerox Research Center Webster in New York. In addition to teaching courses at the University of Kansas, Dr. Agah has taught at the Department of Engineering Systems at the University of Tsukuba in Japan, the Center of Excellence in Remote Sensing Education and Research at Elizabeth City State University in North Carolina, and the Department of Mechanical Engineering at University of Canterbury in New Zealand. He is a senior member of IEEE and ACM.

Dr. Agah received his B.A. in Computer Science with Highest Honors from the University of Texas at Austin, M.S. in Computer Science from Purdue University, and M.S. in Biomedical Engineering and Ph.D. in Computer Science from the University of Southern California.